

Universidad San Carlos de Guatemala
Facultad de ingeniería.
Ingeniería en ciencias y sistemas



Práctica 1:

FilmStars

PONDERACIÓN: 6 pts

Índice

Descripción del problema	3
Alcance del sistema	4
1. Autenticación, Gestión de Sesiones y Multiperfil	4
2. Gestión de Planes y Suscripciones	4
3. Catálogo, Búsqueda y Detalle de contenido	4
4. Sistema de Calificaciones Dinámico	4
5. Servicio Financiero FX-Service con Redis Cache	4
6. Historial de reproducción Reciente	4
7. Sistema de notificaciones por Correo	5
Requisitos y restricciones	5
• Desacoplamiento y Backend políglota:	5
• Código limpio:	5
• Punto de entrada único:	5
• Comunicación de Seguridad de Identidad:	5
• Programación en Base de datos:	5
• Gobierno de código:	5
• Contenedores e infraestructura:	6
• Seguridad de la información:	6
Entregables Requeridos	6
1. Requerimientos del sistema	6
2. Modelo de Casos de Uso (UML)	6
3. Vista de Arquitectura (Modelo 4+1)	6
4. Diagramas Estructurales, Comportamiento y Persistencia	6
5. Entregables de Integración y Pruebas:	7
6. Archivos de configuración:	7
Herramientas permitidas	7
Cronograma	8
Consideraciones	8

Descripción del problema

Una importante cadena de cines desea modernizar su plataforma digital para permitir la venta de boletos en línea. El sistema debe de ser capaz de gestionar picos altos de demanda de manera eficiente garantizando que no ocurran condiciones de carrera al seleccionar asientos de forma simultánea, y evitando la pérdida de transacciones financieras ante caídas del sistema.

Para cumplir con estos atributos de calidad, el departamento de ingeniería ha determinado que el sistema se construirá bajo un enfoque de Arquitectura Orientada a Servicios (SOA). La comunicación crítica entre servicios debe gestionarse obligatoriamente de forma asíncrona, utilizando un middleware de mensajería basado en colas.

Alcance del sistema

La aplicación web debe resolver de extremo a extremo los siguientes módulos y capacidades de negocio:

1. Selección de Ubicación y Funciones

- El usuario debe de poder seleccionar su ciudad para visualizar dinámicamente los cines y los horarios/funciones disponibles en dicha localidad.

2. Visualización de películas por categoría

- Clasificación y despliegue de la cartelera segmentada según el tipo de proyección: Estrenos, Pre-ventas, Re-Estrenos.

3. Selección de Asientos en Tiempo Real

- Mapa interactivo de la sala para la función seleccionada. Se debe mapear visualmente el estado de cada asiento (Disponible, Ocupado, Seleccionado/Bloqueado temporalmente)

4. Flujo de Compra de Boletos

- Procesamiento del flujo desde que se confirma la selección de asientos, se envía la cola de mensajería para su validación, se procesa el pago simulado y se emite el boleto final.

Requisitos y restricciones

Cada dominio conceptual (Usuario, Películas/Cartelera, Reservas/Asientos, Pagos) debe de modelarse como servicio independiente con su propio almacenamiento o lógica de acceso a datos.

Los procesos donde la concurrencia sea crítica deben de desacoplarse mediante un bróker de mensajería. Las peticiones no deben bloquear el hilo principal de la aplicación, en su lugar, deben de entrar a cola y serán procesadas por consumidores independientes.

Entregables Requeridos

1. Requerimientos del sistema

- Requerimientos Funcionales (RF): Listado formal, numerado y priorizado de todas las funciones de la plataforma.
- Requerimientos NO Funcionales (RNF): Especificación cuantitativa de atributos de calidad.

2. Modelo de Casos de Uso (UML)

- Diagrama de Casos de uso de alto nivel
- Primera descomposición: Diagramas detallados por módulo utilizando relaciones de inclusión y extensión.
- Casos de uso expandidos: Documentación de todos los casos de uso identificados con flujos principales, alternativos y de excepción con su propio diagrama local.

3. Vista de Arquitectura (Modelo 4+1 vistas de Krutchten)

- Vista de Escenarios (Casos de uso críticos): Explicación de cómo las vistas técnicas se articulan para resolver los flujos de negocio más complejos (Ej. Compra exitosa concurrente).
- Vista Lógica: Diagrama de paquetes o subsistemas que muestren la organización del diseño de software.
- Vista de Procesos: Definición de hilos, procesos y flujos de comunicación asíncrona, detallando cómo interactúan los productos y consumidores con el Message Broker.
- Vista de Componentes (Desarrollo): Organización de los módulos de código, librerías, interfaces expuestas (REST/gRPC) y dependencias de software.
- Vista de Despliegue (Física): Mapeo de los componentes a la infraestructura de hardware o servicios en la nube (Servidor web, contenedores de Servicios SOA, clúster de RabbitMQ/Kafka, instancias de bases de datos)

4. Diagramas Estructurales, Comportamiento y Persistencia

- Diagrama de Arquitectura General: Esquema conceptual que demuestre la topología SOA elegida, la ubicación del bróker de mensajería y la naturaleza de las conexiones (Síncrona vs Asíncrona).
- Diagrama de componentes: Detalle técnico de nodos físicos/virtuales que soportan el entorno.
- Diagrama de Actividades: Modelado del flujo de trabajo del proceso de negocio del cine, enfocándose en la lógica de selección de asientos, expiración de reservas temporales y confirmación de pago.
- Diagrama de secuencia: Interacción temporal detallada entre los servicios, el bróker de mensajería (colas, exchanges/tópicos), los consumidores y la base de datos para escenario de compra.
- Diagrama Entidad-Relación (ER): Diseño de la base de datos (relacional o no relacional).

5. Entregables de Integración y Pruebas:

- Historial de Git e Integración (Pull Requests): Evidencia en el repositorio del uso correcto de ramas y los Pull Request aprobados para la consolidación del proyecto.
- Creación del Tag con la Versión V1.0.0.

6. Archivos de configuración:

Se deben agregar de forma obligatoria los archivos Dockerfile por servicio y los dos archivos Docker Compose utilizados para desplegar en el entorno local y en la nube.

Herramientas permitidas

Tipo	Categoría	Descripción
Opcional	Lenguajes	Go, TypeScript, Python
Opcional	Framework	FastAPI, Flask, Python, Express, NestJS, Gin
Opcional	Comunicación	RabbitMQ, Kafka
Opcional	Seguridad y Sesiones	JWT, Session Cookies, OAuth
Opcional	Base de datos	MSSQL, MySQL, PostgreSQL, MongoDB
Obligatorio	Control de Versiones	Gitlab
Opcional	Documentación, Diseño, Modelado	Excalidraw, Figma, Canva, LucidChart, Draw.io ,

		StarUML, Visual Paradigm, FossFlow
--	--	---------------------------------------

Cronograma

Tipo	Fecha Inicio	Fecha Fin
Asignación de Proyecto	01/06/2026	03/06/2026
Elaboración	01/06/2026	03/06/2026
Calificación	04/06/2026	05/06/2026

Consideraciones

- **Fecha límite de entrega:** 03 de Junio a las 23:59
- **Nombre del repositorio:** SA_VJ2026_GX
- **Colaborador:** *Samashoas*
- **Medio de entrega:** UEDI
- **Formato de entrega:** Formato Markdown que incluya tabla de integrantes, índice, introducción, desarrollo de todos los diagramas y conclusiones.
- No se permite el uso de herramientas como supabase o prisma para la gestión de la base de datos
- Se deben cargar los archivos crudos de la documentación al repositorio, de lo contrario el diagrama no será válido.